MyCodeSchool

exercițiu 02 — abstractizare

Pagina de tabel știe singură ce tabel deschide · Playwright / TypeScript · Andreea

Unde lucrezi: `sap-starter/`, peste rezolvarea ta de la exercițiul 01 (`TablePage` există și cele 6 clase de tabel o extind).
Nou, ai voie: `abstract class`, `abstract readonly`, o clasă de bază care la rândul ei face `extends`.
Nu atingi: aplicația (`sap-ui/`), `fixtures/test-fixtures.ts`, celelalte specuri în afară de `fr_parent.spec.ts`.
Înainte să începi: rulezi și notezi `npx tsc --noEmit` (fără erori) și `npx playwright test --project=chromium` (`172 passed`). La finalul fiecărui exercițiu trebuie să obții aceleași cifre.

Partea 1 — TablePage devine o pagină SAP

1. TablePage extinde SapTableBrowserPage

nivel 1

Fișierul `pages/TablePage.ts`. Cele 6 tabele sunt afișate în același ecran ca `SapTableBrowserPage`, deci sunt și ele o pagină SAP.

1. Antetul devine `export abstract class TablePage extends SapTableBrowserPage` (importă `SapTableBrowserPage` din `./SapTableBrowserPage.js`).
2. Șterge câmpul `page` și constructorul: le moștenește din `SapTableBrowserPage`.
3. Șterge `filterInput` și `getRowCount`: în `SapTableBrowserPage` există deja, identice.
4. `tableCell(rowIndex, columnName)` rămâne, dar corpul devine `return this.cell(rowIndex, columnName);`

Rezultat așteptat:

```
npx tsc --noEmit                          -> nicio eroare
npx playwright test --project=chromium    -> 172 passed
```

Teorie: `B1-mostenire-extends-super` — constructorul bazei e cel chemat prin `super(page)` din cele 6 clase.

Indiciu: cele 6 clase au deja `super(page)`. Acum `super` ajunge, prin `TablePage`, în constructorul din `SapTableBrowserPage`, care pune `this.page` și toți locatorii ecranului.

Gata când: `TablePage.ts` nu mai are nici constructor, nici `filterInput`; tsc și suita dau cifrele de la început.

De reținut: acum `frParentPage` are și `btnExecute`, `commandInput`, `resultCount`… Un singur obiect pentru un singur ecran.

2. Fiecare tabel își declară numele

nivel 1

1. În `TablePage`, primul rând din clasă: `abstract readonly tableName: string;`
2. În fiecare din cele 6 clase, primul rând din clasă: `readonly tableName = tables.frParent;` (cu cheia potrivită: `itmDetails`, `entitlement`, `mailCount`, `frAudit`, `caseNotes`). Importă `tables` din `../utils/test-data.js`.
3. **Experiment:** șterge rândul `tableName` din `MailCountPage` și rulează `npx tsc --noEmit`. Apoi pune-l la loc.

Rezultat așteptat la experiment:

```
error TS2515: Non-abstract class 'MailCountPage' does not implement
inherited abstract member tableName from class 'TablePage'.
```

Teorie: `C2-polimorfism-abstract` — o clasă abstractă cere ceva de la subclase; compilatorul refuză subclasa care nu dă.

Indiciu: `tableName` nu are valoare în `TablePage`: doar spune „fiecare tabel trebuie să aibă unul". Valoarea o pune fiecare clasă, cu `=`.

Gata când: experimentul dă eroarea de mai sus, iar după ce pui rândul la loc tsc nu mai dă nicio eroare.

De reținut: `abstract` mută greșeala „am uitat" de la rulare (test roșu, poate) la compilare (sigur).

Partea 2 — deschiderea tabelului, scrisă o singură dată

3. Metoda open()

nivel 1

În `TablePage`, sub `tableName`. Importă `transactions` din `../utils/test-data.js`.

```
async open() {
  await this.goTo();
  await this.commandInput.fill(transactions.se16);
  await this.btnOpen.click();
  await this.tableNameInput.fill(this.tableName);
  await this.btnExecute.click();
}
```

Sunt exact cele 5 linii cu care începe fiecare test din `fr_parent.spec.ts`, cu o diferență: `tables.frParent` a devenit `this.tableName`.

Teorie: `C2-polimorfism-abstract` — `Notificare.trimite()`: metoda scrisă o dată în bază folosește ce completează fiecare subclasă.

Indiciu: `this.tableName` e citit abia când se cheamă `open()`, pe un obiect concret (`FrParentPage`, `MailCountPage` …), deci are mereu o valoare.

Gata când: tsc nu dă nicio eroare (testele încă nu folosesc `open()`).

De reținut: `TablePage` nu știe ce tabel deschide, dar știe *cum* se deschide orice tabel.

4. fr_parent.spec.ts fără sapPage

mai greu

Fișierul `tests/fr-parent/fr_parent.spec.ts`, toate testele. Pentru fiecare test:

1. Cele 5 linii de la `sapPage.goTo()` până la `sapPage.tableNameInput.fill(tables.frParent)` devin `await frParentPage.open();`
2. Dacă linia imediat următoare e `sapPage.btnExecute.click()`, o ștergi: `open()` a apăsat deja Execute.
3. Orice alt `sapPage.` devine `frParentPage.`
4. Din parametrii testului scoți `sapPage`: `async ({ frParentPage })`.
5. `tables.frParent` rămas în corp devine `frParentPage.tableName`; din import scoți `tables` și `transactions`.

Înainte / după (testul 7):

```
test('7. Row count correct', async ({ sapPage }) => {
  await sapPage.goTo();
  await sapPage.commandInput.fill(transactions.se16);
  await sapPage.btnOpen.click();
  await sapPage.tableNameInput.fill(tables.frParent);
  await sapPage.btnExecute.click();
  await expect(sapPage.resultCount).toBeVisible();
```

```
test('7. Row count correct', async ({ frParentPage }) => {
  await frParentPage.open();
  await expect(frParentPage.resultCount).toBeVisible();
```

Rezultat așteptat:

```
grep -c "sapPage" tests/fr-parent/fr_parent.spec.ts   -> 0
grep -c "open()" tests/fr-parent/fr_parent.spec.ts    -> 12
npx playwright test --project=chromium               -> 172 passed
```

Teorie: exercițiul 1 din fișa asta — `frParentPage` e acum și pagină SAP, deci are tot ce avea `sapPage`.

Indiciu: fă un test, rulează doar fișierul (`npx playwright test tests/fr-parent --project=chromium`), apoi treci la următorul. Unde filtrul era completat *înainte* de Execute, acum ordinea e: `open()` , completezi filtrul, apeși `btnExecute` .

Gata când: cele trei cifre de mai sus, plus: câte linii avea fișierul înainte și câte are acum (`wc -l`)? Scrie ambele numere.

De reținut: dacă se schimbă cum se deschide un tabel (alt buton, altă tranzacție), modifici `open()` , un singur loc, nu 12 teste.